

Javascript

Promise, static, encapsulation

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Static-Callback</title>
  </head>
  <body>
    <h2>Static</h2>

    <script>
      class Student {
        #salary = 50000;

        display() {
          console.log(this.#salary);
          console.log("Hello there");
        }

        salaryy(data) {
          this.#salary = data;
        }
      }

      // Student.display();
      // console.log(Student.salary);

      // let obj = new Student();

      // console.log(obj.salary);

      // obj.salaryy(60000);
      // obj.display();

      // let result = class Mark {
      //   display() {
      //     console.log("helo class expression");
      //   }
      // };

      // let obj1 = new result();
```

```
// obj1.display();

// callback => function receives
// another function as argument

function greet(name, cb) {
  console.log(name);
  cb(logi);
}

function welcomeMsg(cb1) {
  console.log("Welcome to our page");
  cb1();
}

function logi() {
  console.log("login successfully");
}

greet("Ganesh", welcomeMsg);

let myPromise = new Promise((resolve, reject) => {
  let success = false;

  if (success) {
    resolve("Successfully");
  } else {
    reject("Not successfully");
  }
});

myPromise
  .then((res) => console.log(res))
  .catch((res) => console.log(res));

// promise chaining

function fetchData() {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(1);
    }, 2000);
  });
}

fetchData()
  .then((res) => {
    console.log(res);
  })
}
```

```

        return res * 2;
    })
    .then((res) => {
        console.log(res);
        return res * 2;
    })
    .then((res) => {
        console.log(res);
    });

let resultedValue = fetchData();

console.log(resultedValue);

setTimeout(() => {
    console.log("Set timeout");
}, 5000);

function orderItems(items) {
    return new Promise((resolve) => {
        resolve(`Items ordred ${items}`);
    });
}

//    resolve => then
orderItems("Pizza")
    .then((res) => {
        console.log(res);
        return orderItems("Burger");
    })
    .then((res) => {
        console.log(res);
    })
    .catch((res) => console.log(res));

//    reject => catch
</script>
</body>
</html>

```

Theory

1. Static vs Instance Members

- **Static members** belong to the class itself, not to instances.

- Accessed using `ClassName.method()` instead of `obj.method()`.
- **Private fields (#field)** → encapsulated, accessible only inside the class.
- Example:

```
class Student {
  static school = "FIIT";
  #salary = 50000;
}
```

2. Callbacks

- **Definition:** A function passed as an argument to another function, executed later.
- **Use case:** Handling asynchronous operations before Promises/async-await.
- **Problem:** Can lead to "callback hell" when nested deeply.
- Example:

```
function greet(name, cb) {
  console.log(name);
  cb();
}
```

3. Promises

- **Definition:** Object representing eventual completion/failure of async operation.
- **States:** Pending → Fulfilled → Rejected.
- **Methods:** `.then()`, `.catch()`, `.finally()`.
- **Promise chaining:** Sequential execution of async tasks.
- Example:

```
fetchData()
  .then(res => res * 2)
  .then(res => console.log(res));
```

4. Async Patterns

- **setTimeout** → executes after delay.
- **Promise.all** → runs multiple promises in parallel.
- **Promise chaining** → sequential execution.
- **Async/Await** → cleaner syntax for promises.



Coding Round Questions

- Static fields → Explain difference between static and instance members in classes.
- Private fields → How do private fields (#) work in JavaScript classes?
- Callbacks → Write a program using callbacks to simulate login flow.
- Promise basics → Explain Promise states with example.
- Promise chaining → Show how to multiply values step by step using chaining.
- Promise.all → Demonstrate running multiple promises in parallel.
- Async vs setTimeout → Compare setTimeout with Promises for async tasks.



Machine Coding Round Challenge

Problem: Food Ordering System

Requirements:

- Create a Restaurant class with private field #balance.
- Add static property location.
- Implement orderItem(item) returning a Promise.

- Chain promises to simulate ordering multiple items (Pizza → Burger → Drinks).
- Handle rejection if item is unavailable.
- Use callbacks to simulate user login before placing order.